

Algorithm Design and Analysis Homework 5

Tianyi Guan 2400017423

2026 年 5 月 10 日

1. Problem 7.4

由于 N 是简单容量网络，由定义，其各边容量均为 1。而 f 也是 0-1 可行流，意味着每条边上的流量必为 0 或 1。因此，考虑 $N(f)$ 每条边的计算过程，所有的 $N(f)$ 上的正向边，其容量只可能是 0（当边容量为 1 且真实流量为 1，此时正向边消失）或 1（当边容量为 1 且真实流量为 0）；对于所有 $N(f)$ 上的反向边，其容量只可能是 0（真实流量为 0，此时反向边不生成）或 1（真实流量为 1）。这就证明了 $N(f)$ 中所有存在的边，容量必然均为 1。接下来证明节点结构的限制。对于 N 中的任意中间节点 v ，其原本满足入度为 1 或出度为 1。我们以 v 的入度为 1，出度为 k 为例：

- 当经过 v 的流量为 0 时： $N(f)$ 中与 v 关联的边方向不发生改变，因此在 $N(f)$ 中 v 的入度仍为 1。
- 当经过 v 的流量为 1 时：上游唯一的那条入边必定满流，在 $N(f)$ 中方向反转，转变为 v 的一条出边；而在下游的 k 条出边中，必然有且只有一条边满流，在 $N(f)$ 中方向反转，转变为 v 的一条入边。其余 $k-1$ 条出边流量为 0，在 $N(f)$ 中依然保持为出边。此时清点 $N(f)$ 中 v 的度数可知，其入度仅有那条由反向边生成的入边，总入度依然严格保持为 1。

同理可证，若原网络中 v 的入度为 k ，出度为 1，无论是否有流量经过，经过类似的边反向推导可知，其在 $N(f)$ 中的出度永远保持为 1。

综上所述，在辅助网络 $N(f)$ 中，各边容量均为 1，且所有的中间点依然严格满足或者入度为 1 或者出度为 1 的结构性质。因此， $N(f)$ 仍然是简单容量网络。

2. Problem 7.6

只需要在原有的 N 上加入两个超节点 S^* 和 T^* ，其中 S^* 通过所有出边指向原本的所有发点， T^* 通过所有入边连接原本的所有收点，且所有边容量都为 $+\infty$ 。剩下对这个扩充网络解最大流问题就可以了。

3. Problem 7.7

只需要使用“拆点法”。对于原网络 N 中的每一个带有顶点容量限制的中间节点 $u \in V - \{s, t\}$ ，执行以下操作：将节点 u 拆分成两个全新的节点：入节点 u_{in} 和出节点 u_{out} 。在 u_{in} 和 u_{out} 之间新增一条有向边 $\langle u_{in}, u_{out} \rangle$ ，并将其容量设为原顶点的容量，即 $c(u_{in}, u_{out}) = c(u)$ 。将原网络中所有指向 u 的入边全部改接到 u_{in} 上；所有从 u 出发的出边全部改由 u_{out} 引出。所有原边的容量均保持不变。转化完成后，任何流经原节点 u 的流量都必须且只能经过新增的内部边 $\langle u_{in}, u_{out} \rangle$ ，从而完美受到 $c(u)$ 的物理限制。剩下只需要对这个扩充后的新网络解标准的最大流问题就可以了。

4. Problem 7.17

- (1) 一个简单的想法是直接先调用 dinic 算法求出最大流，然后以这个求出的最大流量作为输入，用 floyd 算法在辅助网络上找负权环路，使得费用最小化。但一直在辅助网络上消圈效率比较低，因为每次 floyd 算法都会带来 $O(V^3)$ 的时间开销。考虑如下的优化算法：

Algorithm 1: 连续最短路算法 (Successive Shortest Path, SSP)**Input:** 带费用的容量网络 $N = (V, E, c, w, s, t)$, 其中 c 为容量, w 为单位费用**Output:** 从源点 s 到汇点 t 的最小费用最大流 F 及最小总费用 C

```

/* 1. 初始化 */
1  $F \leftarrow 0$ ;
2  $C \leftarrow 0$ ;
3  $f(u, v) \leftarrow 0, \quad \forall \langle u, v \rangle \in E$ ;
/* 2. 核心贪心推流主循环 */
4 while True do
5   根据当前流量  $f$ , 构造辅助网络  $N(f)$ ;
   /* 对于  $N(f)$  中的边: 正向边费用为  $w$ , 反向边费用为  $-w$  */
6   使用 SPFA/Bellman-Ford 在  $N(f)$  中寻找从  $s$  到  $t$  的最小费用最短路径  $P$ ;
7   if 找不到路径  $P$  then
8     break;
   ; // 无法继续推流, 已达到最大流
9   else
   /* 3. 计算瓶颈与推流 */
10   $\Delta f \leftarrow \min_{\langle u, v \rangle \in P} \{c_f(u, v)\}$ ;
   ; // 找出路径  $P$  上的最小残余容量
11   $F \leftarrow F + \Delta f$ ;
12   $C \leftarrow C + \Delta f \times \sum_{\langle u, v \rangle \in P} w(u, v)$ ;
   /* 4. 更新实际流量 */
13  foreach 边  $\langle u, v \rangle \in P$  do
14    if  $\langle u, v \rangle$  是原图中的正向边 then
15       $f(u, v) \leftarrow f(u, v) + \Delta f$ ;
16    else
17       $f(v, u) \leftarrow f(v, u) - \Delta f$ ;
   ; // 利用反向边退流
18    end
19  end
20 end
21 end
22 return 最大流量  $F$ , 最小费用  $C$ ;

```

(2) 只需要稍微修改上面的 ssp 算法, 算法伪码如下:

Algorithm 2: 带预算限制的连续最短路算法 (Budget-Constrained SSP)**Input:** 容量网络 N , 边容量 c , 边费用 w , 源点 s , 汇点 t , 总预算 B **Output:** 在费用不超过 B 的前提下的最大流 F

```

1  $F \leftarrow 0$ ;
2  $C \leftarrow 0$ ;
3  $f(u, v) \leftarrow 0, \quad \forall \langle u, v \rangle \in E$ ;
4 while True do
5   根据当前流量  $f$ , 构造辅助网络  $N(f)$ ;
6   使用 SPFA 寻找  $N(f)$  中从  $s$  到  $t$  的最小费用最短路径  $P$ , 设其单位流量费用为  $w_P$ ;
7   if 找不到路径  $P$  then
8     break;
9   else
10     $\Delta f \leftarrow \min_{\langle u, v \rangle \in P} \{c_f(u, v)\}$ ;
11     $B_{\text{remain}} \leftarrow B - C$ ;
12    if  $\Delta f \times w_P \leq B_{\text{remain}}$  then
13      /* 预算充足, 拉满物理瓶颈 */
14       $f_{\text{push}} \leftarrow \Delta f$ ;
15    else
16      /* 预算不足, 提前刹车 (假设允许非整数流, 若不可拆分则向下取整) */
17       $f_{\text{push}} \leftarrow B_{\text{remain}} / w_P$ ;
18    end
19    /* 执行推流、记账与残余网络更新 */
20     $F \leftarrow F + f_{\text{push}}$ ;
21     $C \leftarrow C + f_{\text{push}} \times w_P$ ;
22    更新路径  $P$  上所有正反向边的流量  $f$ ;
23    if  $f_{\text{push}} < \Delta f$  then
24      break;
25    end
26  end
27 return 最大可用流量  $F$ ;

```

5. Problem 7.21 (1)**1. 节点定义:**

- 设立一个超级源点 S 和一个超级汇点 T 。
- 设立四个中间节点 Q_1, Q_2, Q_3, Q_4 , 分别代表四个季度的仓库。

2. 边集与参数定义 (容量 c , 单位费用 w):

第一类: 生产边 (从源点到各季度, 代表当季生产能力与成本) 从 S 向每个 Q_i 连边, 容量为当季最大生产能力, 费用为生产成本:

- $\langle S, Q_1 \rangle$: $c = 25, w = 12.0$
- $\langle S, Q_2 \rangle$: $c = 35, w = 11.0$
- $\langle S, Q_3 \rangle$: $c = 30, w = 11.5$
- $\langle S, Q_4 \rangle$: $c = 20, w = 12.5$

第二类：库存边（相邻季度之间，代表跨季仓储）从 Q_i 向 Q_{i+1} 连边，代表本季未交货的产品存入仓库留至下季，容量无限制，费用为库存费：

- $\langle Q_1, Q_2 \rangle$: $c = +\infty, w = 0.1$
- $\langle Q_2, Q_3 \rangle$: $c = +\infty, w = 0.1$
- $\langle Q_3, Q_4 \rangle$: $c = +\infty, w = 0.1$

第三类：交货边（从各季度到汇点，代表满足当季合同需求）从每个 Q_i 向 T 连边，容量为当季必须交货的数量，由于交货本身不产生额外内部费用，费用设为 0：

- $\langle Q_1, T \rangle$: $c = 15, w = 0$
- $\langle Q_2, T \rangle$: $c = 20, w = 0$
- $\langle Q_3, T \rangle$: $c = 25, w = 0$
- $\langle Q_4, T \rangle$: $c = 20, w = 0$

3. 模型求解：只需要在这个构建好的网络中，求解从 S 到 T 的**总流量为 80 的最小费用流**（或者直接求最小费用最大流，由于汇点入边总容量恰好为 80，最大流必为 80）。求得的最小费用即为最低总成本，每条边上的实际流量即为最优的生产与库存调度计划。